CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 6

Rendering External JSON Data using Fetch API, Search and Filter

Student Name	Shreyas
Roll Number	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To display students, events and FAQs for StudentHub from external JSON files using the JavaScript Fetch API, with dynamic rendering, search, filtering, sorting and pagination, written as modular JavaScript with proper loading and error handling.

- Load three JSON datasets (events, students, FAQs) of at least 15 records each with fetch() and render them as tables.
- Implement search, filter, sort and pagination with JavaScript array methods (filter, some, sort, slice, map).
- Show clear loading and error states, with a Retry button and a reliable way to demonstrate an error.
- Organise the code as ES modules with one responsibility each.
- Advanced extension: cache the latest JSON response in localStorage and show it when loading fails.
- Keep the page accessible and runnable on any Windows computer with Python's built-in web server.

2. Problem Definition

Official problem definition: *"Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination."*

The Practical 6 folder started with a directory page that opened as a blank page: a rule hid the page until a login script ran, and three referenced scripts (auth.js, theme.js and nav.js) were missing. All JavaScript was in one file, the FAQs could not be filtered, sorting was by name only, and there was no caching. The student data used realistic names with roll numbers in the university format and CPI values, and one FAQ named a real person. This practical repairs the page, rewrites the JavaScript as modules and replaces all records with clearly labelled fictional sample data.

The practical list asks four key questions; the answers for this implementation are:

Key question	Answer in this practical
How is JSON fetched, parsed, and rendered?	fetch() requests the file; response.ok is checked; response.json() parses it; the records list is validated; render.js builds the table with createElement and textContent (Sections 6–7).
Which array methods are used for search, filter, sort, and pagination?	filter() with some() and includes() for search, filter() for the dropdown, sort() with localeCompare or subtraction on a copy of the array, and slice() for pages; map() builds rows (Section 8).
How are loading and error states handled?	A loading message with a spinner while fetching; on failure an error with the reason and a Retry button, or the saved localStorage copy with a notice; an 8-second time limit (Section 9).
How is modularity maintained in JavaScript files?	Five ES modules: config (dataset settings), api (fetch and cache), filters (pure functions), render (DOM) and main (connects them) (Section 10).

3. Tools and Technologies

Tool / technology	Use in this practical
HTML5	directory.html: page structure, tabs and controls with ARIA attributes
CSS3	assets/css/style.css: layout, status messages, card layout on small screens; system fonts
JavaScript (ES6+ modules)	Fetch API, async/await, array methods, AbortController, localStorage
JSON	data/students.json, events.json, faqs.json – 15 sample records each
Python 3.12.0 (http.server)	Local web server for running the page; also the static and contrast check scripts
Microsoft Edge 156.0.4314.50	Running and testing the page; headless mode for the scripted tests and screenshots
Visual Studio Code, Git	Source code editor; version control of the StudentHub repository
Operating system	Windows 11

4. Project Structure and How to Run

```
Practical-6/
├── directory.html          page structure (no inline CSS or JS)
├── start-server.bat        Windows: starts the Python server and opens the page
├── assets/css/style.css
├── assets/js/main.js, config.js, api.js, filters.js, render.js, file-check.js
├── data/students.json, events.json, faqs.json
├── screenshots/           Edge screenshots
└── source/check_page.py, contrast_check.py
```

Browsers block fetch() and JavaScript modules for pages opened directly from a folder (file://), so the page must be served by a local web server. Python's built-in server is used; PHP is not needed.

- **Option A:** double-click start-server.bat. It starts the server and opens http://127.0.0.1:8606/directory.html.
- **Option B:** in Command Prompt, in the Practical-6 folder, run python -m http.server 8606 --bind 127.0.0.1 and open http://127.0.0.1:8606/directory.html.
- **Port 8606:** on the development PC, port 8000 was already used by a XAMPP PHP server, so a separate port and the 127.0.0.1 address are used to avoid reaching the wrong server.
- **Python check:** the launcher uses the first Python that actually runs, because on the development PC the py launcher pointed to a removed Python version.
- **Opened as a file:** file-check.js (a classic script) replaces the page with these instructions instead of showing a broken page.
- **Other computers:** all paths are relative, so copying the Practical-6 folder and installing Python 3 is enough.

5. JSON Datasets (Sample Data)

Each file is an object that labels itself as sample data and holds a records list of 15 objects:

```
{ "dataset": "students", "sampleData": true,
  "description": "SAMPLE DATA - fictional students created for ITUE203 Practical 6. ...",
  "records": [ {"id": "DEMO-001", "name": "Lavanya Rao", "branch": "CE", "semester": 7, "cpi":
8.37}, ... ] }
```

File	Records	Fields
students.json	15	id (DEMO-001 to DEMO-015), name, branch (CSE/CE/IT), semester, cpi – all fictional
events.json	15	id, title, date (ISO yyyy-mm-dd), venue, category (7 categories) – example events
faqs.json	15	id, category (Academics, Attendance, Campus, Exams, Fees, Hostel, Portal), question, answer

- **Privacy:** all original student names, roll numbers and CPI values were replaced with invented names, DEMO IDs and invented CPI values. The static check confirms that no university-format roll number (such as 25CS023) appears in any data file.
- **No named individuals:** the FAQ that named the HOD now gives a role-based answer ("The Head of Department (HOD). Contact the department office for current details."). Answers that stated specific university rules now point to the official regulations.
- **Visible label:** the page shows a sample-data notice, and each table caption says "sample data".

6. Fetch API and JSON Parsing

fetchDataset() in api.js requests a file, checks the HTTP status, parses the JSON and checks that it contains a records list. Every failure becomes an Error with a readable message, which main.js shows to the user. The request has an 8-second time limit (AbortSignal.timeout) combined with a cancel signal from main.js. For demonstrations, simulateError requests a file that does not exist, so the real 404 error path is shown.

6.1 api.js – fetchDataset()

```
15  // Fetches one dataset and checks the response. Throws an Error with a readable message on failure.
16  // options.simulateError requests a file that does not exist, so the real error path can be
    demonstrated.
17  export const fetchDataset = async (url, { signal, simulateError = false, delayMs = 0 } = {}) => {
18      if (delayMs > 0) await wait(delayMs, signal);
19      const target = simulateError ? url.replace(/\.json$/, '-missing.json') : url;
20      const timeout = AbortSignal.timeout(TIMEOUT_MS);
21      const combined = signal && AbortSignal.any ? AbortSignal.any([signal, timeout]) : (signal ||
    timeout);
22
23      let response;
24      try {
25          response = await fetch(target, { signal: combined, cache: 'no-store' });
26      } catch (error) {
27          if (error.name === 'AbortError' && signal?.aborted) throw error;          // cancelled by a
    newer request
28          if (error.name === 'TimeoutError') throw new Error(`${target} took longer than ${TIMEOUT_MS /
    1000} seconds`);
29          throw new Error(`the network request for ${target} failed`);
30      }
31      if (!response.ok) throw new Error(`the server answered ${response.status} ${response.statusText}
    for ${target}`);
32
33      let body;
34      try {
35          body = await response.json();
36      } catch {
37          throw new Error(`${target} is not valid JSON`);
38      }
39      if (!body || !Array.isArray(body.records)) throw new Error(`${target} has no "records" list`);
40      return body;
41  };
```

7. Dynamic Rendering

render.js builds the table from the column definitions in config.js. Values are inserted with textContent, never innerHTML, so text from the JSON files can never be interpreted as HTML. Column labels are readable (“Student ID”, “CPI”), dates are formatted (22 Aug 2026), CPI is shown with two decimals, each cell carries a data-label used by the phone layout, the sorted column receives aria-sort, and search matches are wrapped in <mark>.

7.1 render.js – renderTable()

```
30  export const renderTable = (container, config, items, { query, sort, pageInfo }) => {
31      const caption = el('caption', {
32          text: `${config.label} – sample data (page ${pageInfo.page} of ${pageInfo.totalPages})`,
33      });
34      const headRow = el('tr', {}, config.columns.map((column) => {
35          const th = el('th', { scope: 'col', text: column.label });
36          if (column.numeric) th.className = 'num';
37          if (sort?.key === column.key) th.setAttribute('aria-sort', sort.dir === 1 ? 'ascending' :
    'descending');
38          return th;
39      }));
40      const rows = items.map((record) => el('tr', {}, config.columns.map((column) => {
41          const raw = record[column.key];
42          const shown = column.format ? column.format(raw) : String(raw);
43          const searchable = config.searchKeys.includes(column.key);
44          return el('td', { 'data-label': column.label, className: column.numeric ? 'num' : '' },
45              searchable ? highlight(shown, query) : [shown]);
46      })));
47      container.replaceChildren(el('table', { className: 'data-table' }, [caption, el('thead', {},
    [headRow]), el('tbody', {}, rows)]));
48  };
```

8. Search, Filter, Sorting and Pagination

Feature	Array logic (filters.js)	Behaviour
Search	filter() + some() + includes() on lower-case text	Case-insensitive, spaces trimmed; only the listed fields are searched (e.g. Student ID, name, branch), so hidden numeric ids of events and FAQs no longer cause false matches; matches highlighted
Filter	filter() with equality on one field	Branch (students) or category (events and FAQs); options built from the data with Set
Sort	sort() on a copy: subtraction for numbers, localeCompare for text	Students: name, CPI, semester · Events: date, title · FAQs: question, category; ISO dates sort correctly as text
Pagination	slice(start, start + perPage)	5, 10 or 15 rows; Previous, page numbers, Next; current page clamped; reset to page 1 when the search, filter or sort changes

The functions never change the original array, so main.js simply applies search → filter → sort → paginate to the loaded records every time a control changes (update(), Section 8.2).

8.1 filters.js – search, filter, sort and paginate

```
// Search, filter, sort and pagination as pure functions: they never change the original array,
// so the same records can be re-filtered every time a control changes.

const text = (value) => String(value).toLowerCase();

// Search: keep records where any of the listed fields contains the query (case-insensitive).
export const searchRecords = (records, query, keys) => {
    const q = text(query.trim());
    if (!q) return records;
    return records.filter((record) => keys.some((key) => text(record[key]).includes(q)));
};

// Filter: keep records whose field equals the chosen value ('all' keeps everything).
export const filterRecords = (records, key, value) =>
    value === 'all' ? records : records.filter((record) => String(record[key]) === value);

// Sort: numbers numerically, everything else alphabetically (ISO dates sort correctly as text).
export const sortRecords = (records, sort) => {
    if (!sort || !sort.key) return records;
    const { key, dir } = sort;
    return [...records].sort((a, b) => {
        const x = a[key];
        const y = b[key];
        const result = typeof x === 'number' && typeof y === 'number'
            ? x - y
            : String(x).localeCompare(String(y), 'en', { sensitivity: 'base' });
        return result * dir;
    });
};

// Pagination: returns the records for one page plus the numbers needed for the controls.
export const paginate = (records, page, perPage) => {
    const totalPages = Math.max(1, Math.ceil(records.length / perPage));
    const current = Math.min(Math.max(1, page), totalPages);
    const start = (current - 1) * perPage;
    return {
        items: records.slice(start, start + perPage),
        page: current,
        totalPages,
        from: records.length ? start + 1 : 0,
        to: Math.min(start + perPage, records.length),
    };
};
```

8.2 main.js – update()

```
33  // Applies search, filter, sort and pagination to the loaded records and redraws the view.
34  const update = (focusAction = null) => {
35      const config = DATASETS[state.name];
36      const sort = config.sorts.find((s) => s.value === sortSelect.value);
37      const matches = sortRecords(
38          filterRecords(searchRecords(state.records, searchInput.value, config.searchKeys),
    config.filter.key, filterSelect.value),
39          sort,
40      );
41      const pageInfo = paginate(matches, state.page, Number(pageSizeSelect.value));
42      state.page = pageInfo.page;
43
44      if (!matches.length) {
45          renderEmpty(tableArea, config, () => {
46              searchInput.value = '';
47              filterSelect.value = 'all';
48              state.page = 1;
49              update();
50              searchInput.focus();
51          });
52          summary.textContent = `No ${config.plural} found (0 of ${state.records.length}).`;
53      } else {
54          renderTable(tableArea, config, pageInfo.items, { query: searchInput.value, sort, pageInfo });
55          const source = state.source === 'live' ? `live data from ${config.url}` : 'saved copy
    (localStorage)';
56          summary.textContent = `Showing ${pageInfo.from}–${pageInfo.to} of ${matches.length} $
    {config.plural}`
57              + `${matches.length < state.records.length ? ` (filtered from ${state.records.length})` :
    ''} · ${source}`;
58      }
59      renderPagination(pagination, pageInfo, (page, action) => {
60          state.page = page;
61          update(action);
62      });
63      if (focusAction) {                                                          // keep keyboard
    focus in the pagination
64          const target = pagination.querySelector(`[data-action="${focusAction}"]:not(:disabled)`)
65              || pagination.querySelector('[aria-current="page"]');
66          target?.focus();
67      }
68  };
```

8.3 config.js – the students dataset

```
 7  export const DATASETS = {
 8      students: {
 9          label: 'Students',
10          plural: 'students',
11          url: 'data/students.json',
12          columns: [
13              { key: 'id', label: 'Student ID' },
14              { key: 'name', label: 'Name' },
15              { key: 'branch', label: 'Branch' },
16              { key: 'semester', label: 'Semester', numeric: true },
17              { key: 'cpi', label: 'CPI', numeric: true, format: (v) => v.toFixed(2) },
18          ],
19          searchKeys: ['id', 'name', 'branch'],
20          filter: { key: 'branch', label: 'Branch', all: 'All branches' },
21          sorts: [
22              { value: 'default', label: 'Student ID (default)' },
23              { value: 'name-asc', label: 'Name (A–Z)', key: 'name', dir: 1 },
24              { value: 'name-desc', label: 'Name (Z–A)', key: 'name', dir: -1 },
25              { value: 'cpi-desc', label: 'CPI (high to low)', key: 'cpi', dir: -1 },
26              { value: 'cpi-asc', label: 'CPI (low to high)', key: 'cpi', dir: 1 },
27              { value: 'semester-asc', label: 'Semester (1 to 7)', key: 'semester', dir: 1 },
28          ],
29      },
```

9. Loading, Error Handling, Retry and localStorage Cache

- **Loading:** while a request runs, a “Loading students…” message with a spinner is shown, the panel has aria-busy="true" and the controls are disabled.
- **Error:** if loading fails and no saved copy exists, an error message (role="alert") gives the reason – for example “the server answered 404 File not found for data/students-missing.json” – with a Retry button.
- **Cache (advanced extension):** every successful response is saved in localStorage with the time. If a later request fails, the saved copy is shown with the notice “Showing a saved copy from …” and a Retry button. All localStorage calls are wrapped in try/catch, because storage can be full or blocked.
- **Other errors handled:** invalid JSON, JSON without a records list, server errors such as 500, and the 8-second time-out.
- **Testing tools:** a panel at the bottom of the page can simulate a loading error (a real 404), simulate a slow network (3 seconds) and clear the saved copies, so each state can be demonstrated reliably.

9.1 api.js – localStorage cache

```
43  // Cache helpers: localStorage can be full or blocked, so every call is wrapped in try/catch.
44  export const saveToCache = (name, body) => {
45      try {
46          localStorage.setItem(CACHE_PREFIX + name, JSON.stringify({ savedAt: new Date().toISOString(),
    body }));
47          return true;
48      } catch {
49          return false;
50      }
51  };
52
53  export const readFromCache = (name) => {
54      try {
55          const cached = JSON.parse(localStorage.getItem(CACHE_PREFIX + name));
56          return cached && Array.isArray(cached.body?.records) ? cached : null;
57      } catch {
58          return null;
59      }
60  };
```

9.2 main.js – load()

```
// and the previous request is cancelled, so quickly switching tabs can never show the wrong data.
const load = async (name) => {
    const requestId = ++latestRequest;
    controller?.abort();
    controller = new AbortController();
    const config = DATASETS[name];
    state.name = name;
    state.records = [];
    filterLabel.textContent = config.filter.label;                                  // correct label while loading

    setBusy(true);
    tableArea.replaceChildren();
    pagination.replaceChildren();
    summary.textContent = '';
    renderStatus(status, { type: 'loading', title: `Loading ${config.plural}…`, message: `Fetching ${config.url}` });

    try {
        const delayMs = slowToggle.checked ? SLOW_NETWORK_MS : Number(params.get('delay')) || 0;
        const body = await fetchDataset(config.url, { signal: controller.signal, simulateError: errorToggle.checked, delayMs });
        if (requestId !== latestRequest) return;
        saveToCache(name, body);
        renderStatus(status);
        showRecords(body.records, 'live');
    } catch (error) {
        if (requestId !== latestRequest || error.name === 'AbortError') return;
        const cached = readFromCache(name);
        if (cached) {
            renderStatus(status, {
                type: 'cached',
                title: `Showing a saved copy from ${formatTime(cached.savedAt)}`,
                message: `The latest ${config.plural} could not be loaded: ${error.message}.`,
                onRetry: () => load(name),
            });
            showRecords(cached.body.records, 'cache');
        } else {
            const hint = window.location.protocol === 'file:' ? ' Open the page through the local server (see README.md).' : '';
            renderStatus(status, {
                type: 'error',
                title: `Could not load the ${config.plural}`,
                message: `Reason: ${error.message}.${hint}`,
                onRetry: () => load(name),
            });
            summary.textContent = `No ${config.plural} to show.`;
        }
    } finally {
        if (requestId === latestRequest) setBusy(false);
    }
};
```

10. Modular JavaScript and Race-Condition Guard

Module	Responsibility
config.js	Dataset definitions: URL, columns and labels, searchable fields, filter field, sort options. A new dataset needs only a new entry.
api.js	fetchDataset() and the localStorage cache (saveToCache, readFromCache, clearCache).
filters.js	Pure functions: searchRecords, filterRecords, sortRecords, paginate, uniqueValues.
render.js	DOM output: renderTable, renderEmpty, renderPagination, renderStatus, fillSelect.
main.js	Entry module (type="module"): state, tabs, controls, loading and error handling.
file-check.js	Classic script that explains how to open the page when it is opened as a file.

Race condition: if a user switches tabs while a slow request is still running, the older response could arrive last and overwrite the new tab. load() therefore numbers each request and ignores any response that is not the latest, and it cancels the previous request with AbortController (Section 9.2, lines 98–100, 115 and 120). The tests confirmed that a deliberately slowed Students response is ignored after switching to Events.

10.1 main.js – imports

```
1  // StudentHub – Practical 6: entry module. Connects the tabs and controls to the data, filter and render
   modules.
2  import { DATASETS, PAGE_SIZES, SLOW_NETWORK_MS } from './config.js';
3  import { fetchDataset, saveToCache, readFromCache, clearCache } from './api.js';
4  import { searchRecords, filterRecords, sortRecords, paginate, uniqueValues } from './filters.js';
5  import { renderTable, renderEmpty, renderPagination, renderStatus, fillSelect } from './render.js';
```

11. Accessibility

- **Tabs:** WAI-ARIA tabs pattern – role="tablist"/"tab"/"tabpanel", aria-selected, aria-controls and one tab in the Tab order; Left/Right arrows, Home and End move between tabs.
- **Controls:** search, filter, sort and rows-per-page each have a visible label; the filter label changes to Branch or Category.
- **Announcements:** the status area and the results summary (“Showing 1–5 of 15 students”) are live regions; errors use role="alert".
- **Table:** caption, th with scope="col", aria-sort on the sorted column; pagination buttons have aria-label and aria-current, and focus stays on the pagination after a page change.
- **Contrast and focus:** all 22 checked colour pairs meet WCAG 2.1 AA; a visible violet focus outline.

11.1 directory.html – tabs

```
37  <div class="tabs" role="tablist" aria-label="Dataset">
38      <button type="button" role="tab" id="tab-students" aria-controls="data-panel" aria-selected="true"
    data-dataset="students">Students</button>
39      <button type="button" role="tab" id="tab-events" aria-controls="data-panel" aria-selected="false"
    tabindex="-1" data-dataset="events">Events</button>
40      <button type="button" role="tab" id="tab-faqs" aria-controls="data-panel" aria-selected="false"
    tabindex="-1" data-dataset="faqs">FAQs</button>
41  </div>
42
43  <section class="panel" id="data-panel" role="tabpanel" aria-labelledby="tab-students">
```

12. Browser Screenshots

All screenshots are genuine renders of directory.html in Microsoft Edge 156 (headless), served by Python’s http.server, produced with the same scripted clicks and typing used in the tests. To keep them readable, the report shows the part of the page from the tabs down (Figure 1 shows the complete page). The full images are in Practical-6/screenshots/.

12.1 JSON data displayed

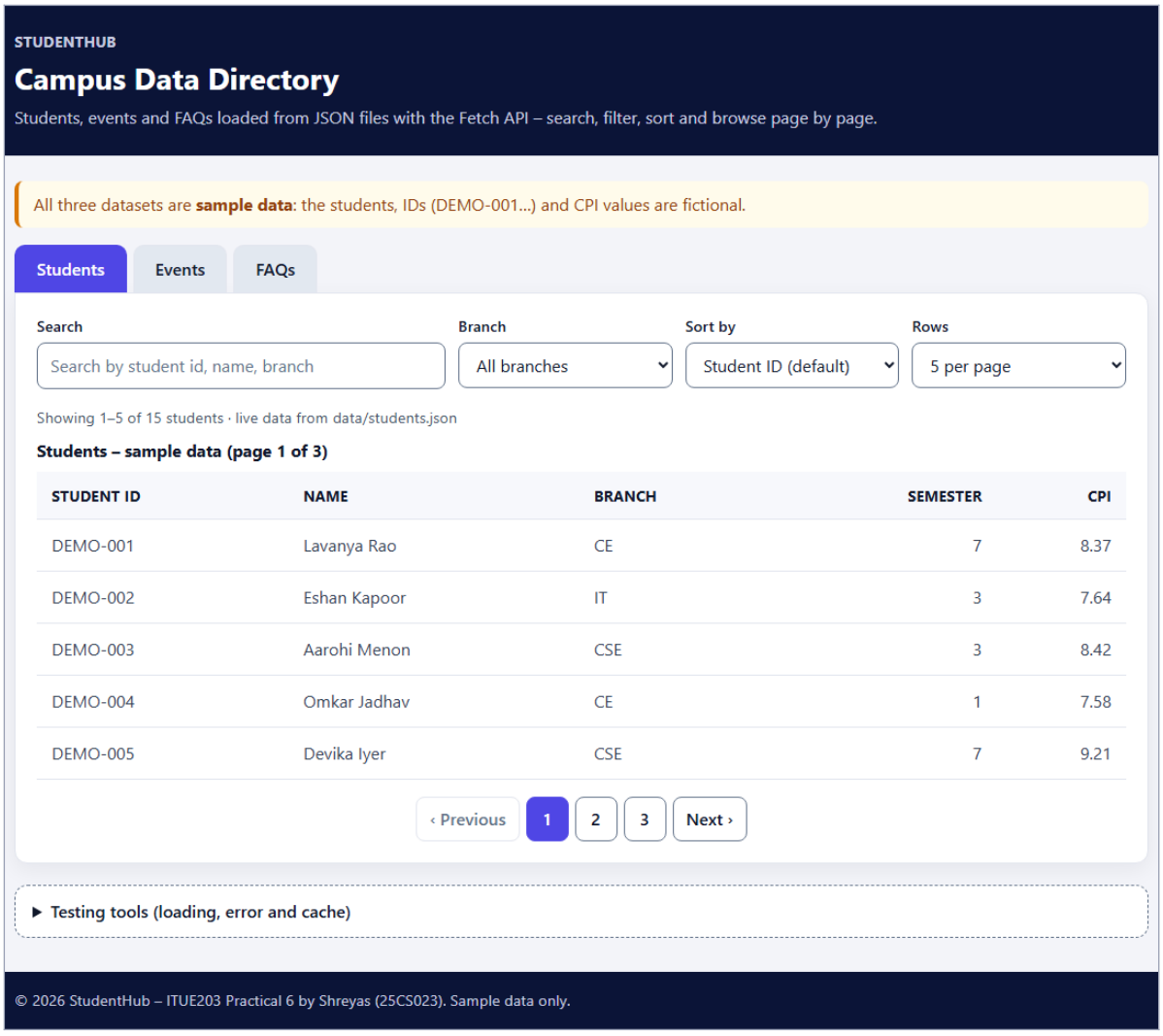


Figure 1: Students loaded from data/students.json – page 1 of 3, live data, fictional sample records – json-data.png

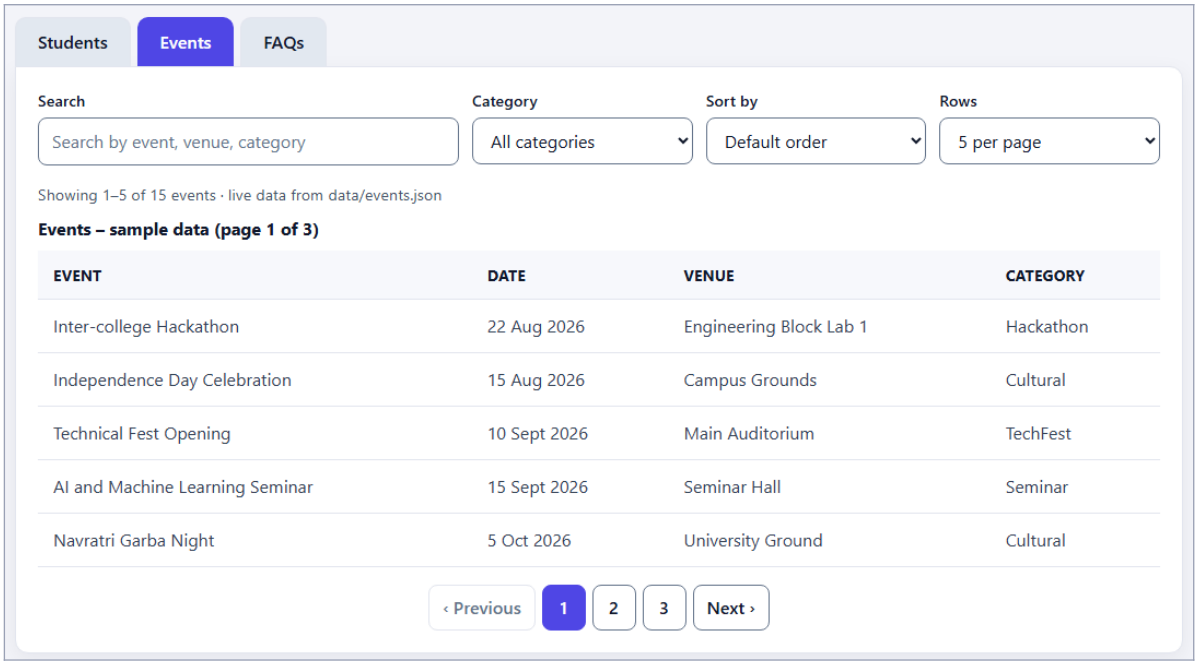


Figure 2: Events dataset – readable column labels and formatted dates – events-data.png

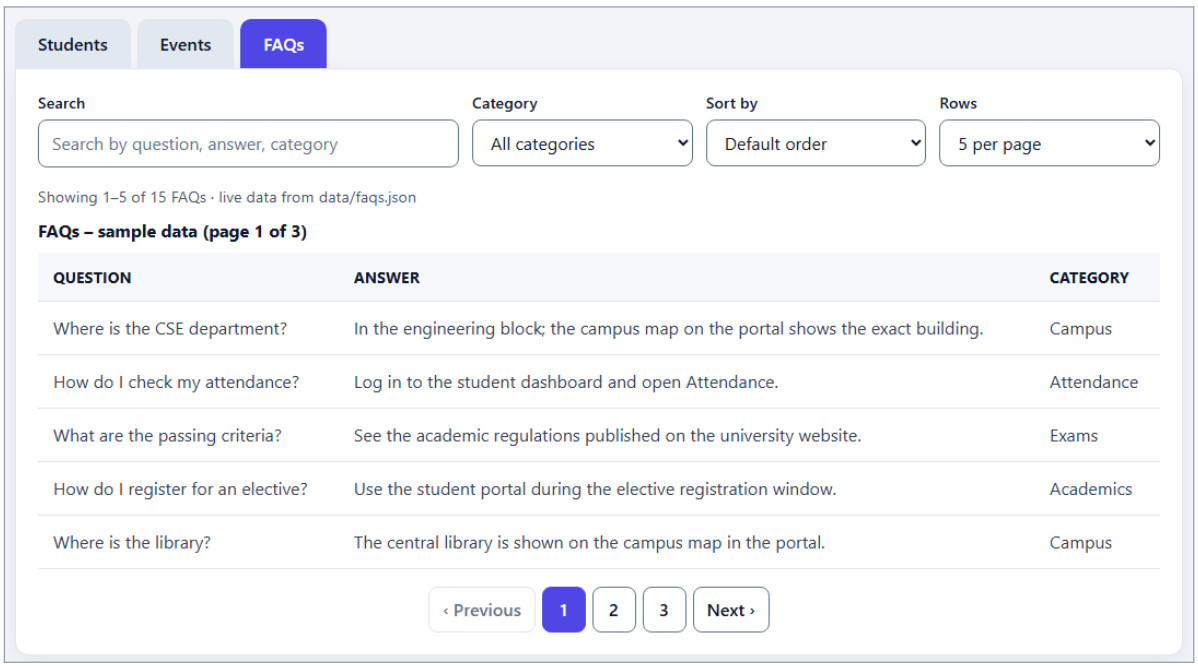


Figure 3: FAQs dataset with the new Category column – faqs-data.png

12.2 Search result

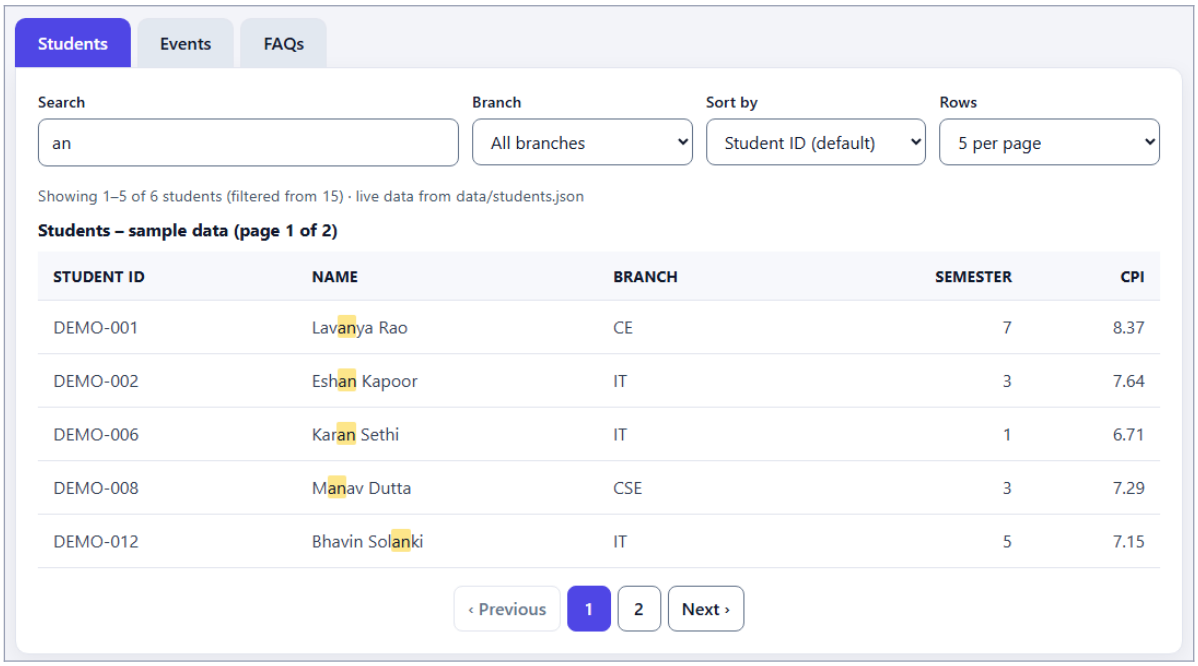


Figure 4: Search “an” – 6 of 15 students match, highlighted with <mark>, shown on 2 pages – search.png

12.3 Filter result

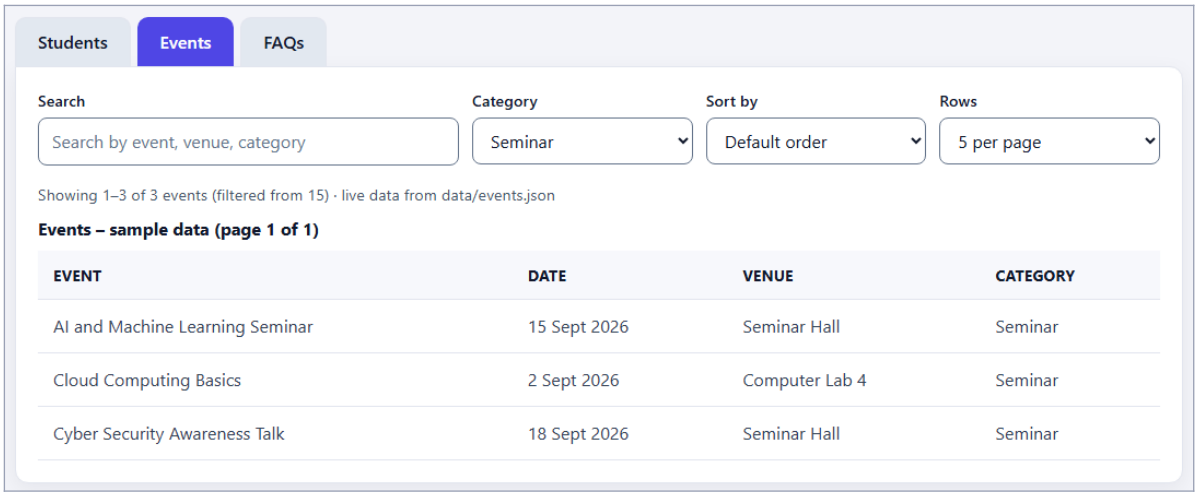


Figure 5: Events filtered by category Seminar – 3 of 15 events – filter.png

12.4 Sorting result

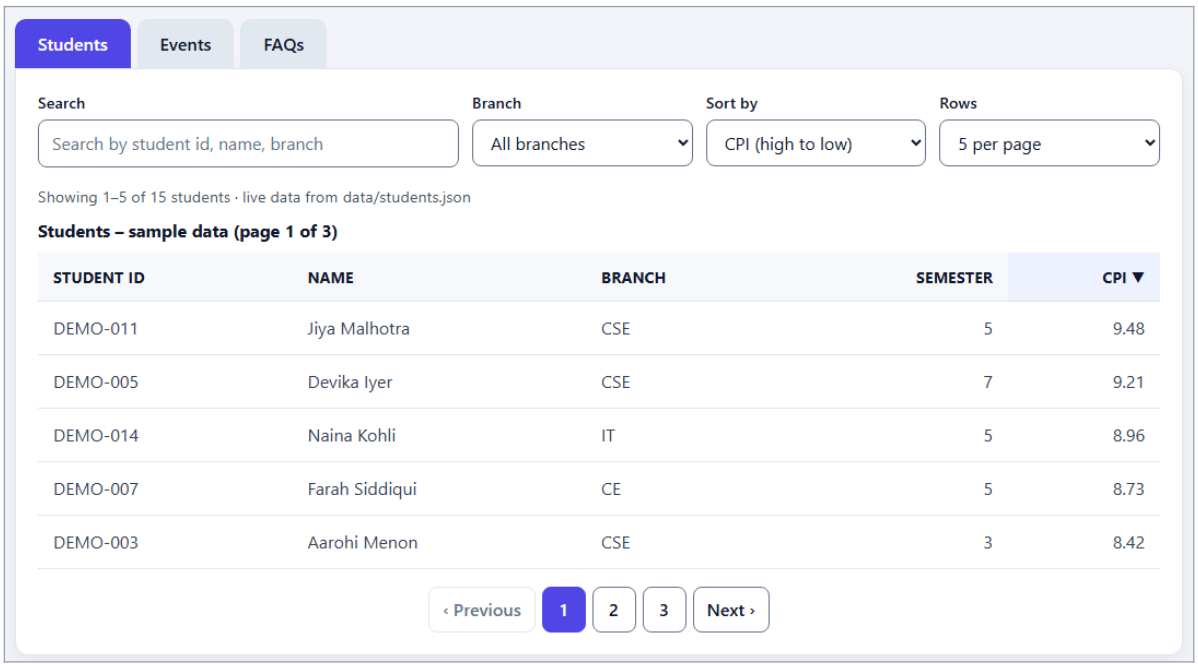


Figure 6: Students sorted by CPI, high to low – the CPI header shows ▼ (aria-sort=descending) – sort.png

12.5 Pagination

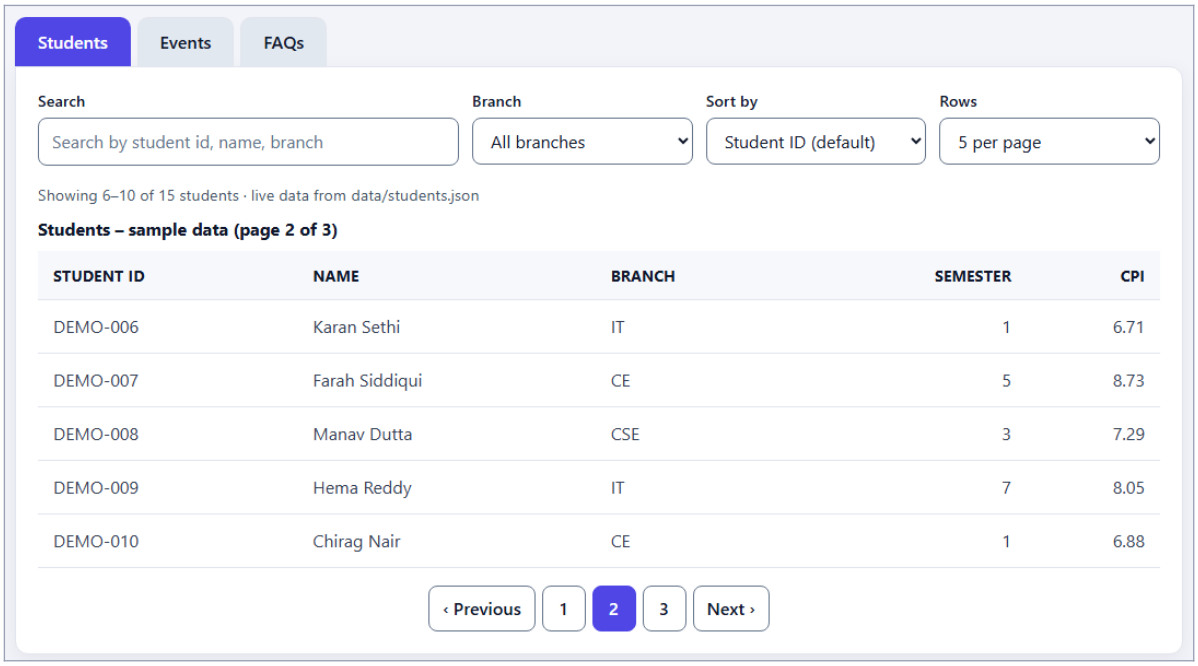


Figure 7: Page 2 of 3 – DEMO-006 to DEMO-010, "Showing 6–10 of 15 students" – pagination.png

12.6 Loading and error messages

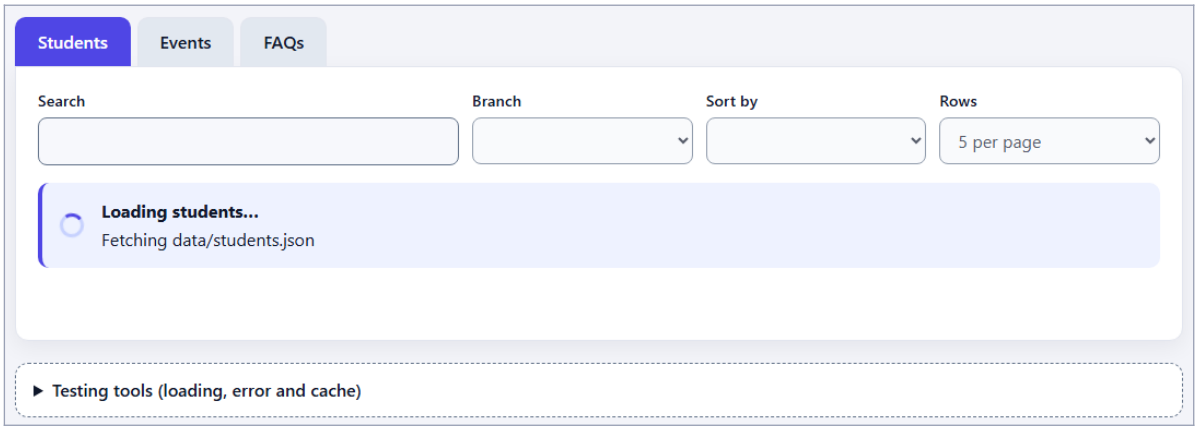


Figure 8: Loading message with spinner while data/students.json is being fetched; controls disabled – loading.png

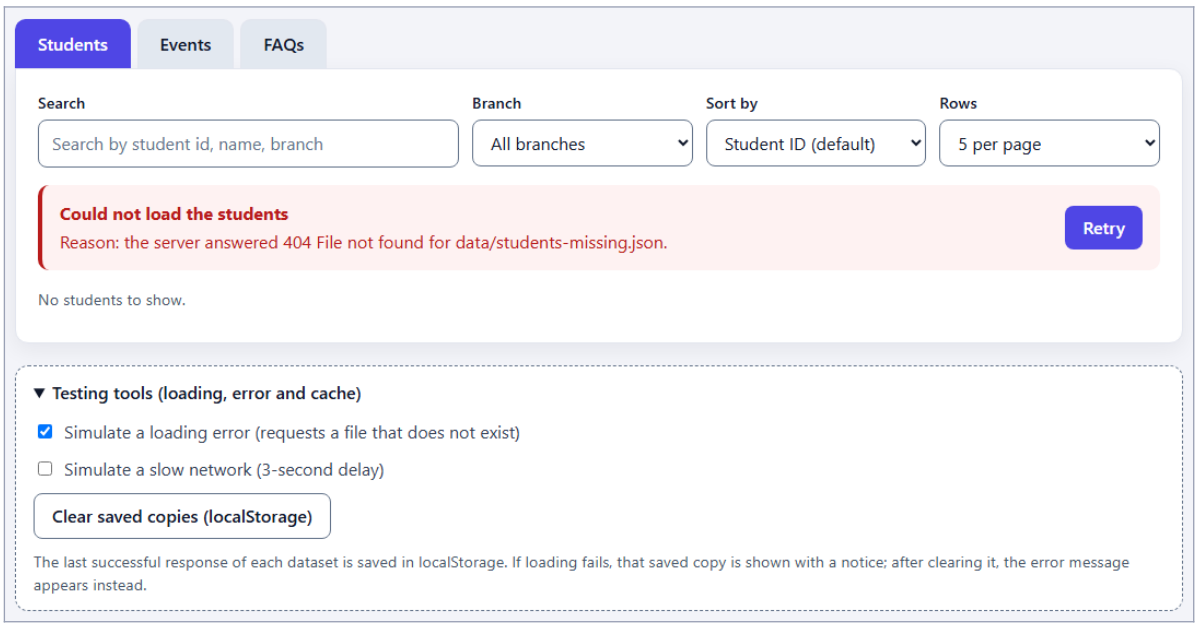


Figure 9: Error after "Simulate a loading error" with no saved copy – reason (404 for data/students-missing.json) and Retry – error.png

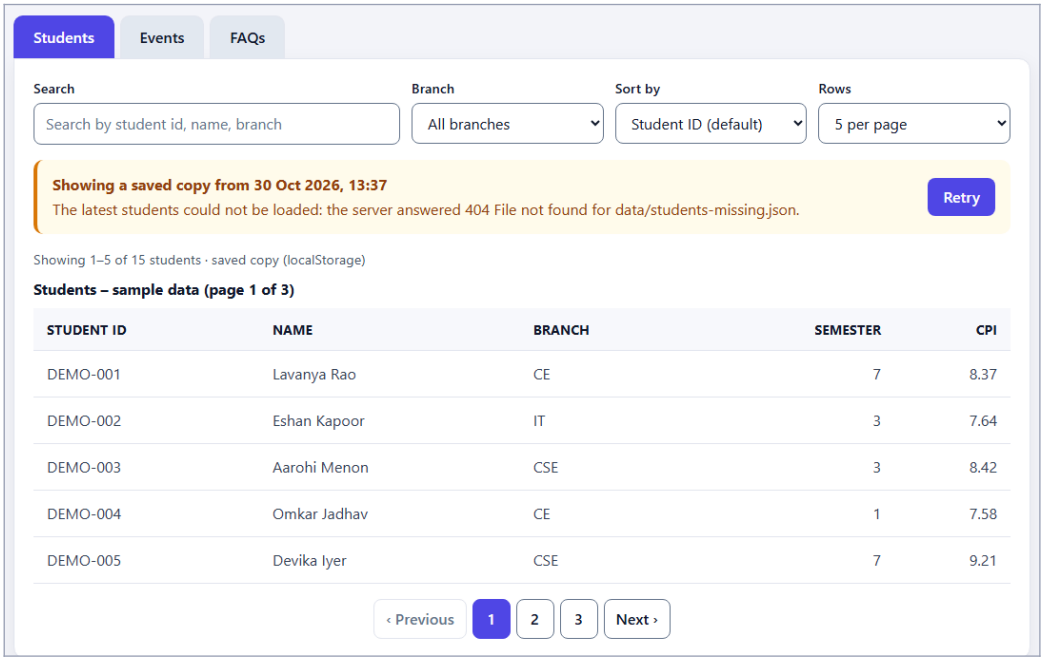
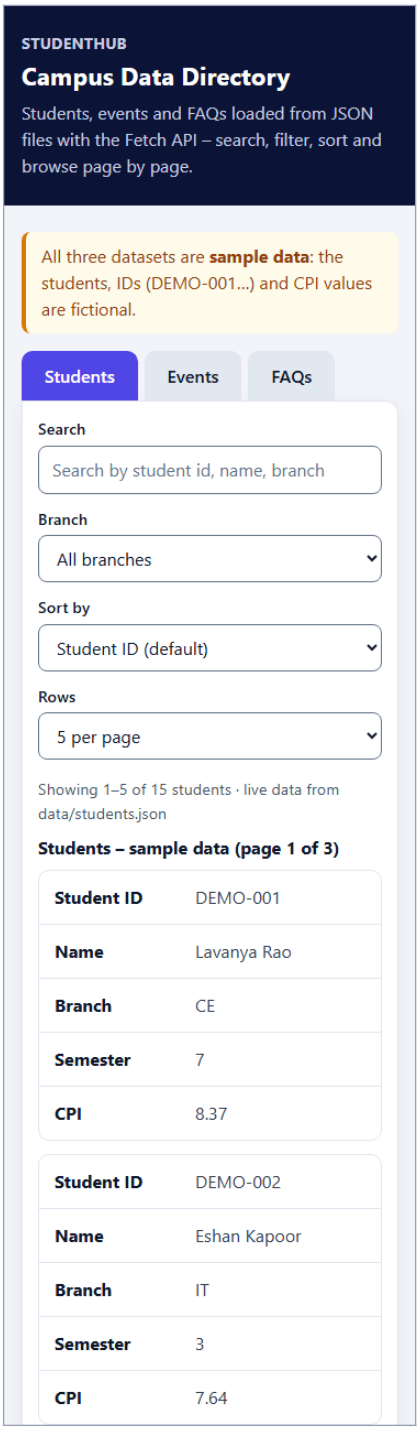
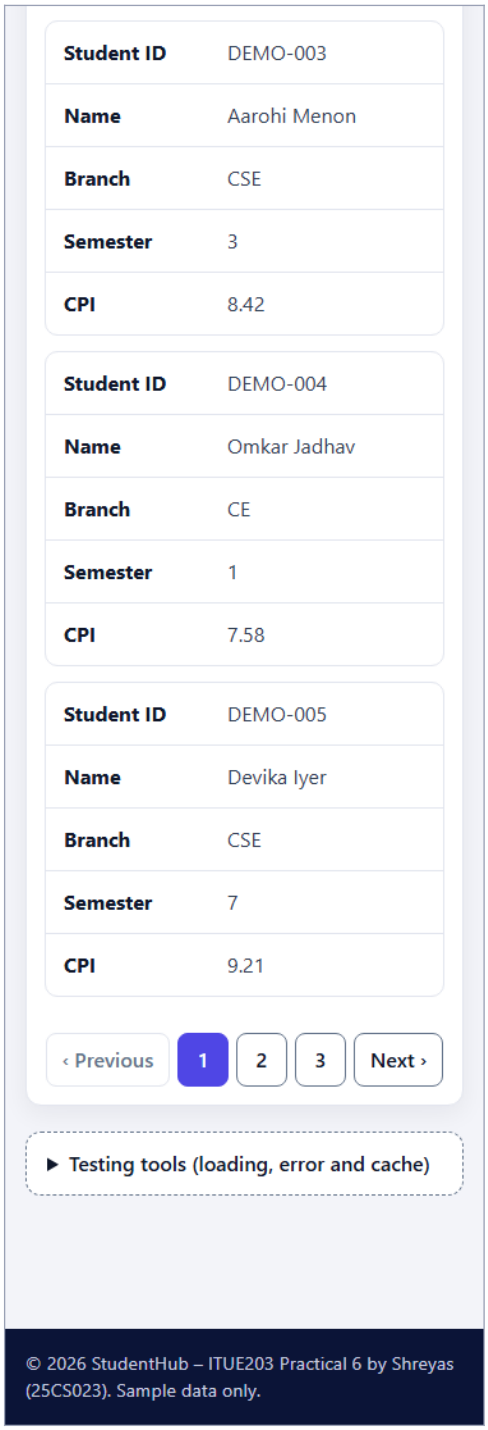


Figure 10: Loading failed but a saved copy exists – data shown from localStorage with a notice and Retry – cached-copy.png

12.7 Phone layout (375px)



(a) Top part



(b) Bottom part

Figure 11: At 375px each record becomes a card with labels; no horizontal scrolling – mobile-375.png, shown in two parts

13. Testing Results

13.1 Static checks – source/check_page.py

A Python script written for this practical (standard library only, not the W3C validator). Result: 29 of 29 checks passed.

#	Check	Result
1–5	No inline style, inline handlers, <style> or inline script; no external URLs; all referenced files exist	Pass
6–8	main.js loaded as a module; every import resolves; code split into config, api, filters, render and main	Pass
9–15	No duplicate ids, one h1; ids used by the JavaScript exist; ARIA references valid; tab pattern; labelled controls; buttons typed; live regions	Pass
16–23	Each JSON file labelled as sample data with 15 records, same fields and unique ids; DEMO IDs; no university-format roll numbers; no named individuals; config columns exist in the data; FAQ categories	Pass
24–29	fetch with response.ok and records validation; localStorage in try/catch; request-id and AbortController guard; no innerHTML; no var; start-server.bat uses http.server with Windows line endings	Pass

An Edge check also imported each library module on its own (api, config, filters, render) to confirm that it parses and its imports resolve; Node.js was not available for a separate syntax check.

13.2 Colour contrast – source/contrast_check.py

22 colour pairs from style.css were checked with the WCAG 2.1 formula (4.5:1 for text, 3:1 for borders and focus outline). All 22 pass; the lowest are placeholder text and disabled page buttons at 4.76:1.

13.3 Edge tests

Method: a temporary Python script served Practical-6 with http.server and opened test copies of directory.html in Microsoft Edge 156 (headless). A script in each copy clicked, typed and changed the real controls and checked the DOM, focus and localStorage. A small wrapper around fetch() could add a delay or return a broken response for one file, to test loading, time-outs, malformed JSON and fast tab switching. Page errors, unhandled promise rejections and console.error were recorded in every case. A control run with deliberately false checks confirmed that failures are reported. Result: 93 of 93 checks passed, with no console errors.

Test case	What was checked	Result
All three datasets	Page visible; first request fetch(data/students.json); readable labels; DEMO IDs; CPI with 2 decimals; caption; summary; events dates; FAQs; 15 records each; role-based HOD answer	13 / 13
Search	Highlighting; case-insensitive; extra spaces; Student ID; “cse” (5); “an” (6, 2 pages); no-match message and Clear; FAQs “fees” (2); events “workshop” (2)	11 / 11
Filter	Branch options; CE (5); IT + “kohli” (1); CE + “kohli” (empty); events label and 7 categories; Seminar (3); FAQs Attendance (3)	8 / 8
Sorting	CPI both ways; aria-sort; name both ways; semester; default order; event date both ways; FAQ question	11 / 11

Test case	What was checked	Result
Pagination	3 pages; Next, page 3, Previous; aria-current; focus kept; reset to page 1 after search; 10 and 15 per page	9 / 9
Loading	Loading message, spinner, aria-busy, disabled controls; cleared after loading; slow-network option	6 / 6
Time-out	Still loading at 7 s; error “took longer than 8 seconds” with Retry	3 / 3
Error and cache	Response saved; Clear saved copies; simulated 404 with role=alert and reason; Retry sends a new request; recovery; saved copy shown after failure; Retry on notice	10 / 10
Bad responses	Malformed JSON; JSON without records; server 500; Retry after the fix	5 / 5
Fast tab switching	Slow Students response ignored after switching to Events; no error for the cancelled request; rapid three-tab switching ends on FAQs	4 / 4
Keyboard	One tab in the Tab order; ArrowRight, End, Home, ArrowLeft (wrap); labels; scope=col; live regions	8 / 8
375px	No horizontal scrolling (Students and FAQs); rows shown as labelled cards	4 / 4
Opened as file://	Server instructions shown and the app hidden	1 / 1

Each case except file:// also checked for page and console errors (included in the counts).

13.4 Problems found and fixed during testing

- **Python launcher:** on the development PC the py command pointed to a removed Python 3.14, so start-server.bat failed. It now uses the first Python that actually runs (python, then py -3).
- **Port conflict:** a XAMPP PHP server already used port 8000, so a browser opening localhost:8000 could reach the wrong server. The practical now uses port 8606 and the address 127.0.0.1.
- **Loading screenshot:** the first loading screenshot showed an error, because the test’s slow-network wrapper reported the app’s 8-second time-out as a plain cancellation. The wrapper was fixed, a time-out test was added, and the screenshot was retaken while the page was still loading.
- **Search highlight:** side padding on <mark> made highlighted words look split (“Lav an ya”); the padding was removed.

13.5 Not tested

- **Manual use:** nobody operated the page with a physical mouse and keyboard; events were dispatched by script, and real Tab-key movement was not simulated.
- **Screen readers, other browsers, real phones:** not tested; only Microsoft Edge on Windows 11 was used, and 375px used a narrow browser frame.
- **W3C validation:** the page was not submitted to the W3C Markup Validation Service.
- **start-server.bat:** tested with its nobrowser option (the server started and served the page, JSON and modules); the automatic opening of the browser was not tested.

14. Results and Conclusion

The StudentHub campus data directory now loads students, events and FAQs from three external JSON files with the Fetch API and renders them as accessible tables. Search, filter, sorting and pagination work for all three datasets using array methods, and the code is organised as five ES modules. Loading, error and empty

states are handled with clear messages and a Retry button; the advanced extension caches the latest response in localStorage and shows it, with a notice, when loading fails. A request guard prevents stale responses when switching tabs quickly.

All data is clearly labelled fictional sample data, with no real student names, roll numbers or CPI values and no named individuals. The page runs on any Windows computer with Python through start-server.bat or one command. The static checks (29 of 29), contrast checks (22 of 22) and Edge tests (93 of 93, with no console errors) all passed, and the screenshots show every required state.

15. References

- ITUE203 Web Development Frameworks – Practical List 2026-27, Practical 6: “Rendering External JSON Data using Fetch API, Search and Filter”, CSPIT, CHARUSAT.
- MDN – Using the Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN – JavaScript modules: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules
- MDN – AbortController: https://developer.mozilla.org/en-US/docs/Web/API/AbortController
- MDN – Window.localStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- W3C WAI-ARIA Authoring Practices – Tabs pattern: https://www.w3.org/WAI/ARIA/apg/patterns/tabs/
- Python documentation – http.server: https://docs.python.org/3/library/http.server.html
- StudentHub repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023